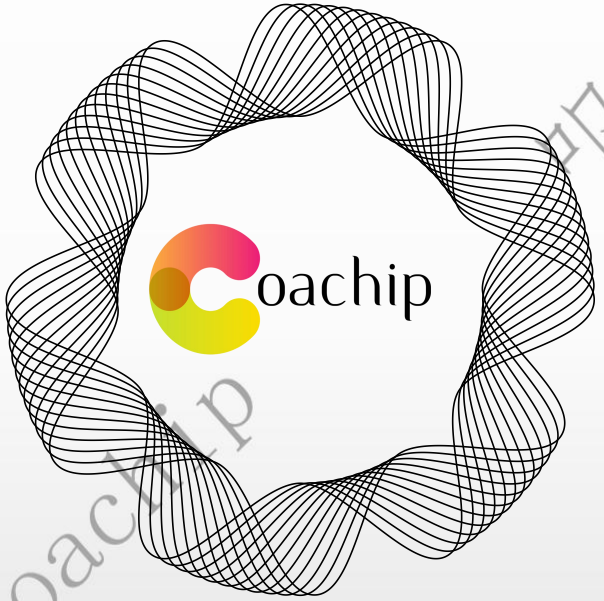


SV的激励时序

叩特电子Coachip

课程内容及目标

课程内容：

- 竞争冒险
  - 程序块(Program Block)和时序区域(Timing Region)
  - 使用clocking block来进行信号的同步

目标：

- 了解竞争冒险
- 了解程序块和时序区域
- 如何进行信号同步

叩特电子Coachip

竞争冒险现象

- 竞争(Competition)：

在组合逻辑电路中，某个输入变量通过两条或两条以上的途径传到输出端，由于每条途径延迟时间不同，到达输出门的时间就有先有后，这种现象称为竞争。

- 冒险(risk)：

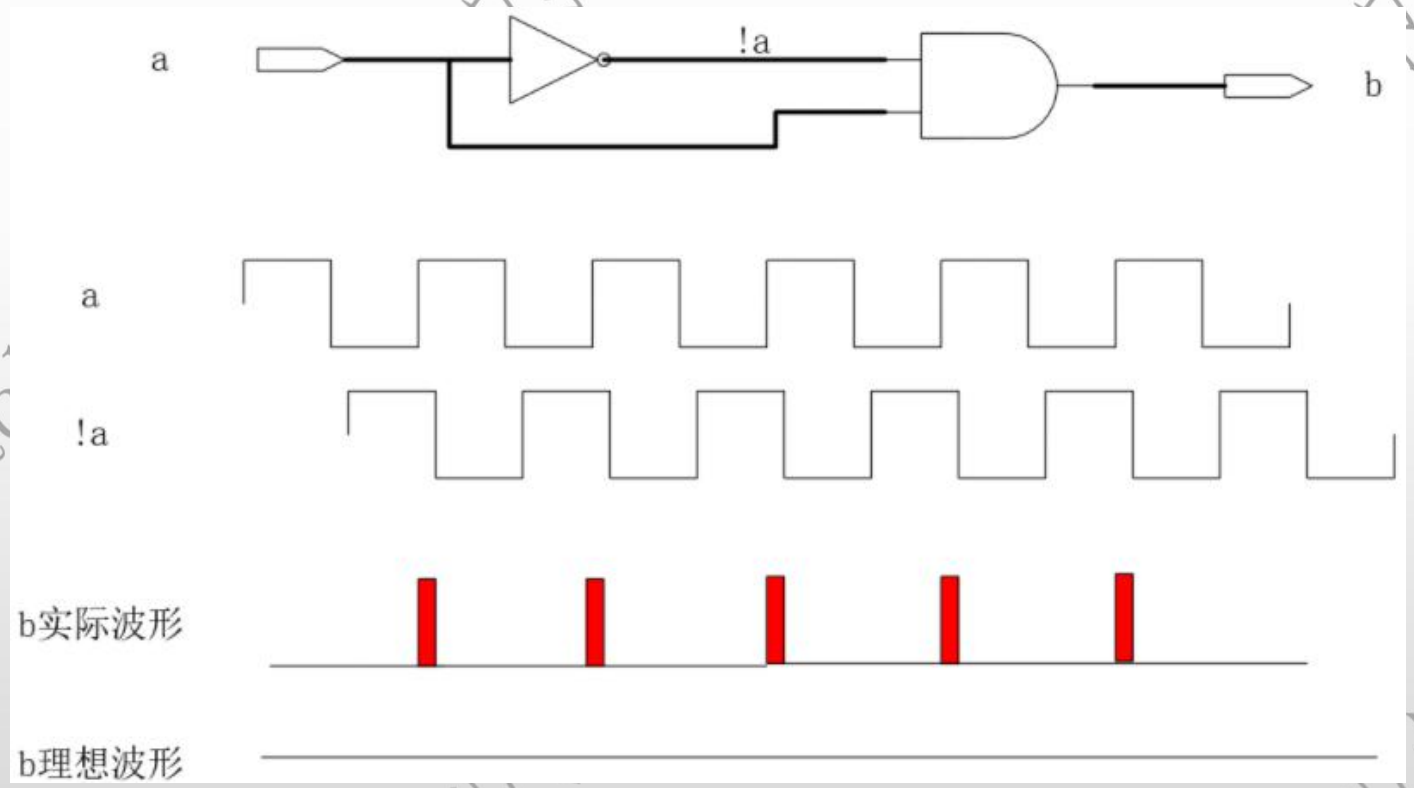
信号在器件内部通过连线和逻辑单元时，都有一定的延时。延时的大小与连线的长短和逻辑单元的数目有关，同时还受器件的制造工艺、工作电压、温度等条件的影响。信号的高低电平转换也需要一定的过渡时间。由于存在这两方面因素，多路信号的电平值发生变化时，在信号变化的瞬间，组合逻辑的输出有先后顺序，并不是同时变化，往往会出现一些不正确的尖峰信号，这些尖峰信号称为"毛刺"。如果一个组合逻辑电路中有"毛刺"出现，就说明该电路存在冒险。

叩特电子Coachip

竞争冒险现象

- 竞争冒险(Competition risk)产生原因：

由于延迟时间的存在，当一个输入信号经过多条路径传送后又重新会合到某个门上，由于不同路径上门的数目不同，或者门电路延迟时间的差异，导致到达会合点的时间有先有后，从而产生瞬间的错误输出。



即特电子Coachip

竞争冒险现象

```
module design_race;
logic clk;
logic rstn;
logic [3:0] a;
logic [3:0] b;
... //省略了时钟和复位产生代码

always_ff @(posedge clk) begin
      a = 0;
      a = a + 1;
      b = a;
      $display("@%0t a=%0d, b=%0d", $time, a, b);
  end
endmodule
```

叩特电子Coachip

竞争冒险现象

```
module memory( input wire start, write,
               input wire [7:0] addr,
               input wire [7:0] data);
  logic [7:0] mem[256];

  always@(posedge start) begin
    if(write)
      mem[addr]=data;
    …
  end
endmodule
```

```
module test( output logic start, write,
             output logic [7:0] addr, data
           );
initial begin
  start = 0; // Initialize signals
  write = 0;
  #10; // Short delay
  addr = 8'h42; // Start first command
  data = 8'h5a;
  start = 1;
  write = 1;
...
end
endmodule
```

设计和测试有时会对同一信号进行访问，例如：testbench正在初始化某一变量，而rtl同时要读这个变量，思考一下读回的是初始化前的值还是后的值？

叩特电子Coachip

竞争冒险现象

在处理仿真中的竞争冒险问题时，一般采用加#0延时，如：

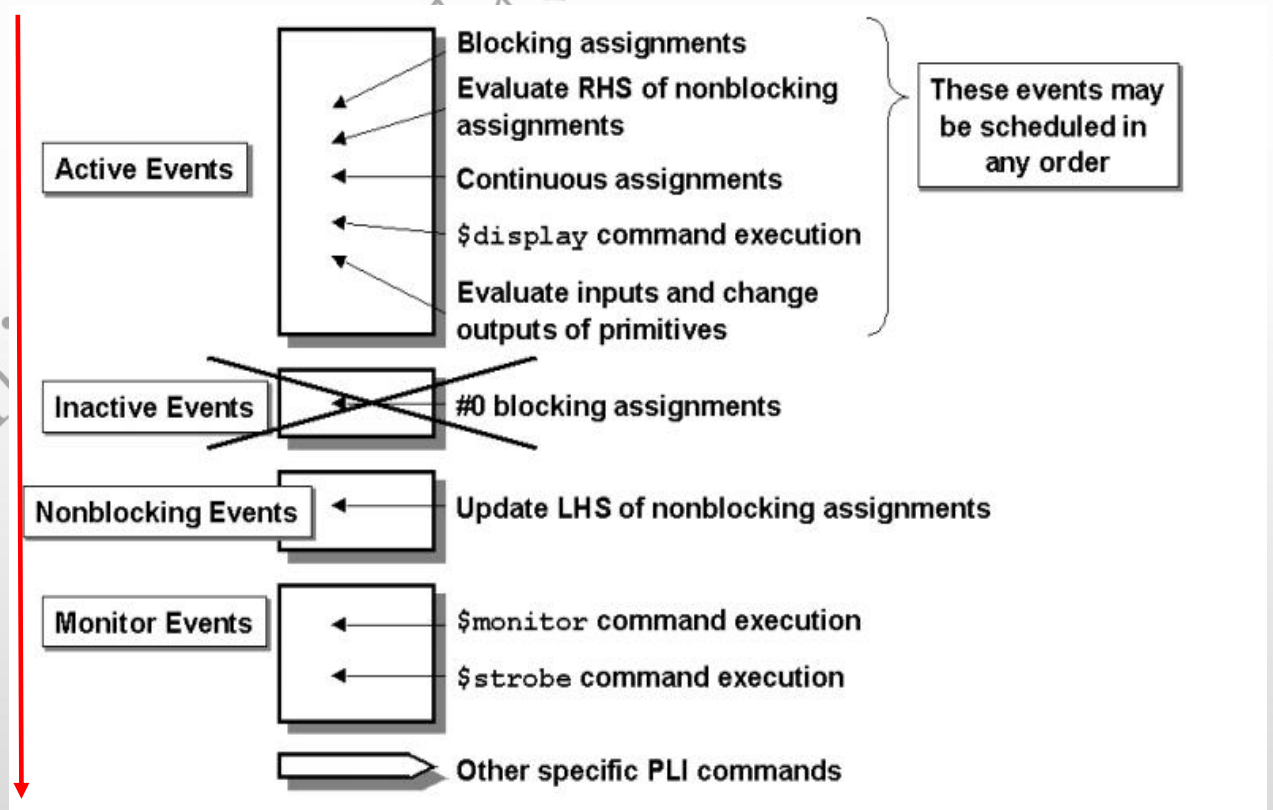
```
a  =  b;
#0 c  =  a;
```

- 在同一个time slot，#0的作用是指定其后面的语句后执行，例如上述语句，a=b先执行，c=a后执行，但若是多个#0延时语句，编译时就无法判定谁先执行了。
- 加上#1，也是不好的，因为在不同的module中时间精度不同的话，就不能区分#1所代表的时间。
- 为了避免在RTL仿真行为中发生的信号竞争问题，我们建议通过非阻塞赋值或者特定的信号延迟来解决同步的问题。

Coachip confidential

层次化事件队列

仿真时，仿真器对一个仿真时刻（timeslot）内语句的执行是按照一定的事件顺序执行的，即所有的事件都是按照一定的队列执行，这个队列在Verilog和SystemVerilog称之为层次化事件队列（Stratified event queue）。在Verilog中层次化事件队列如下，为了尽可能的避免验证环境和设计的竞争冒险，SystemVerilog对原有的Verilog层次化事件队列进行了丰富。

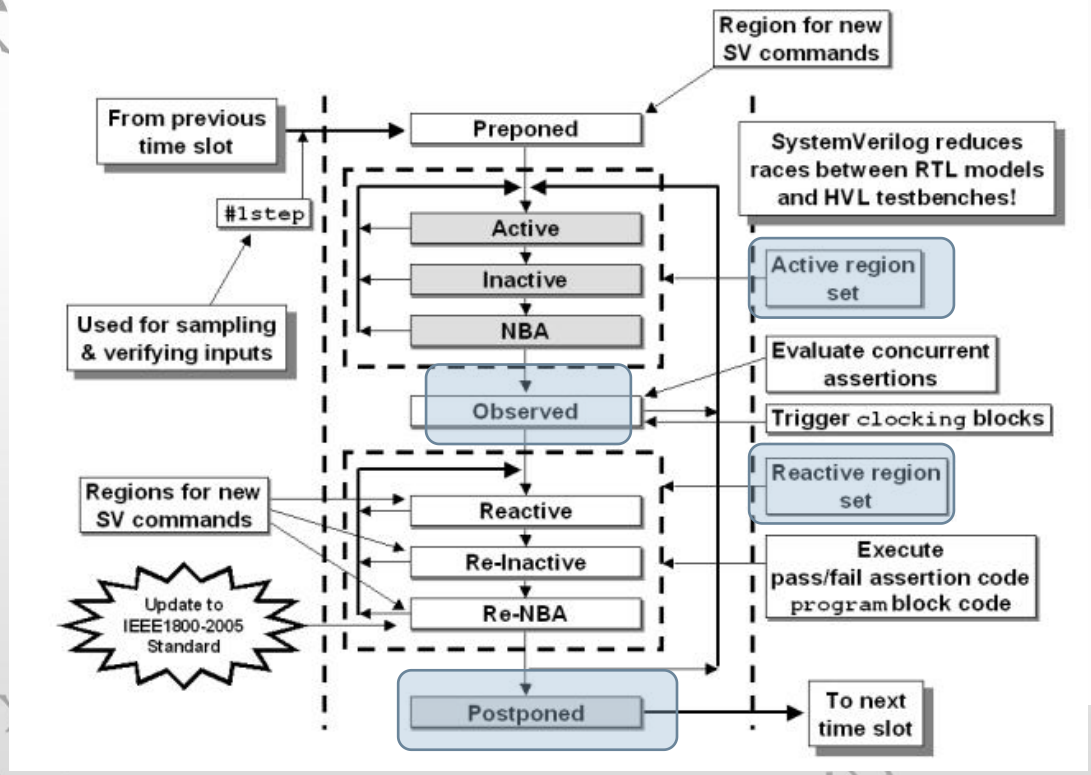


叩特电子Coachip

程序块(Program Block)和时序区域(Timing Region)

SV中约定了一种schedule，使RTL代码和验证代码在同一时刻(time slot) 操作，按照一定的顺序执行。

SV中，为了区分验证和RTL事件，testbench代码都包含在program模块中，program同module的形式很类似，只是它不包含任何的层次结构，也就是说， program内部不能再包含module, interface, program。



Name	Activity
Active	Simulation of design code in modules
Observed	Evaluation of SystemVerilog Assertions
Reactive	Execution of testbench code in programs
Postponed	Sampling design signals for testbench input

使用clocking block来进行信号的同步

在clocking block，所有信号的采样和驱动，都是跟时钟同步的。

```
interface arb_if(input bit clk);
        logic [1:0] grant, request;
        logic reset;
        clocking cb @(posedge clk); // 声明一个时钟模块cb
         output request;
         input grant;
        endclocking
        modport TEST (clocking cb, // 调用cb
                      output reset);
        modport DUT (input request, reset, output grant)
endinterface
```

叩特电子Coachip

SV的激励时序– 本章回顾

- 竞争冒险
- 程序块(Program Block)和时序区域(Timing Region)
- 使用clocking block来进行信号的同步

叩特电子Coachip

Q&A

昂特电子Coachip